ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Data Type Design — Class Challenge Set

The domain: a blockchain ledger

A **blockchain** is, at its core, an append-only ledger of transactions. Unlike a traditional bank, where all transactions are simple transfers of money, a blockchain supports different *kinds* of transactions with fundamentally different structure and semantics. In this challenge we model a simplified ledger with three kinds of transactions:

- A **Transfer** moves a certain amount of cryptocurrency from one address to another. It is the simplest transaction: it has a sender, a receiver, and an amount.
- A **ContractDeploy** publishes a new smart contract to the blockchain. A smart contract is a program that lives on the chain and can be called by other transactions. A deploy has an owner (the deployer), the bytecode of the contract (which we represent as a `String` for simplicity), and a gas limit — the maximum computational effort the deployer is willing to pay for.
- A **ContractCall** invokes a function on an already-deployed contract. It has a caller, the address of the target contract, the name of the function being called, and a gas limit. The actual computational cost (the *gas cost*) depends on what the call does; the gas limit must be sufficient to cover it, or the transaction is invalid.

Each kind of transaction has different fields, different fee structures, different validation rules, and produces a different human-readable description. This makes the domain a natural fit for studying how to design data types with variant-specific operations.

The only exception is the *timestamp* attribute, which is common to all transactions: it records the time when the transaction was executed / stored on the ledger.

Part A — The Haskell datatype

Complete the following Haskell module by providing implementations for all listed functions.

```haskell
module Ledger (Transaction(..), Ledger, fee, gasCost, isValid,
               describe, totalFees, filterByAddress,
               txInRange, groupByKind) where

data Transaction
  = Transfer       { from :: String, to :: String, amount :: Double, timestamp :: Int }
  | ContractDeploy { owner :: String, bytecode :: String, gasLimit :: Int, timestamp :: Int }
  | ContractCall   { caller :: String, contract :: String, function :: String,
                     gasLimit :: Int, timestamp :: Int }
  deriving Show

type Ledger = [Transaction]

-- | Transaction fee
-- Transfer:       1% of amount
-- ContractDeploy: gasLimit * 0.000001  (gas price in crypto units)
-- ContractCall:   gasLimit * 0.000001

-- | Estimated computational cost (in gas units)
-- Transfer:       21000  (fixed, minimal computation)
-- ContractDeploy: 32000 + 200 * length of bytecode  (storing code costs gas per byte)
-- ContractCall:   50000 + 100 * length of function name  (base cost + dispatch overhead)
gasCost :: Transaction -> Int

-- | Validity check
-- Transfer:       amount > 0 and from /= to
-- ContractDeploy: bytecode is non-empty and gasLimit >= gasCost
-- ContractCall:   function name is non-empty and gasLimit >= gasCost
isValid :: Transaction -> Bool

-- | Human-readable one-line description
-- Transfer:       "Transfer: alice -> bob 50.0 @1000"
-- ContractDeploy: "Deploy: alice deploys contract (300000 gas) @1001"
-- ContractCall:   "Call: bob calls swap on 0xABC123 (150000 gas) @1002"
describe :: Transaction -> String
```

```haskell
-- | Total fees across all transactions in the ledger
totalFees :: Ledger -> Double
-- map each transaction to the fee, and then sum

-- | All transactions involving a given address
-- (as sender, receiver, owner, caller, or target contract)
filterByAddress :: String -> Ledger -> Ledger

-- | All transactions with timestamp in [lo, hi]
txInRange :: Int -> Int -> Ledger -> Ledger

-- | Partition into (transfers, deploys, calls)
groupByKind :: Ledger -> ([Transaction], [Transaction], [Transaction])
```

Some example values

```haskell
t1 = Transfer "alice" "bob" 50.0 1000
t2 = ContractDeploy "alice" "0x6060604052..." 300000 1001
t3 = ContractCall "bob" "0xABC123" "swap" 150000 1002
t4 = Transfer "bob" "carol" 30.0 1003
t5 = ContractCall "alice" "0xABC123" "stake" 200000 1004
t6 = Transfer "carol" "alice" 10.0 1005

myLedger = [t1, t2, t3, t4, t5, t6]
```

Key decisions to discuss as you work:

- is the proposed datatype definition effective to represent the transaction structure?
- `totalFees` : which higher-order list function is the right tool — `map` , `filter` , `foldr` , or a combination?
- `filterByAddress` : a `Transfer` has *two* addresses (`from` , `to`), a `ContractDeploy` has one (`owner`), a `ContractCall` has two (`caller` , `contract`). The predicate logic is genuinely different per variant.
- `groupByKind` : can you do this with a single `foldr` ? What is the accumulator type?

Part B — OO design in Scala

Transaction operations

Translate `Transaction` into Scala using the OO style. Complete the case classes by providing constructor parameters and method implementations:

```
sealed trait Transaction:
  def timestamp: Int
  def fee: Double
  def gasCost: Int
  def isValid: Boolean
  def describe: String

case class Transfer(...) extends Transaction:
  ...

case class ContractDeploy(...) extends Transaction:
  ...

case class ContractCall(...) extends Transaction:
  ...
```

Observation: In the Haskell module, each of `fee`, `gasCost`, `isValid`, `describe`, is one function with three clauses. In the Scala OO style, it is three separate method implementations. Which organisation is easier to read? Which is easier to extend with a new variant?

Ledger operations as package functions

Not all operations fit as methods on `Transaction`. Operations like `totalFees`, `filterByAddress`, and `groupByKind` act on the ledger (a collection), not on a single transaction. In Scala, these live as standalone functions in the package:

```
package ledger

// ... trait and case classes as above ...

def totalFees(txs: List[Transaction]): Double = ...
def filterByAddress(addr: String, txs: List[Transaction]): List[Transaction] = ...
def groupByKind(txs: List[Transaction]): (...) = ...
```

Implement these three functions.

Extensions and comparisons

1. **Add a new operation** `isExpensive: Boolean` that returns `true` if the fee exceeds 100.0. Where does it go: method on the trait, or package function? What do you have to modify?
2. **Add a new variant** On a blockchain, a token is a digital asset (a currency, a collectible, a voting right). Minting is the act of creating a new token by registering it on the chain with a name and an initial supply. Add a new variant to your Scala code, to represent a new transaction type associated with tokens:

```
TokenMint(owner: String, tokenName: String, supply: Int, gasLimit: Int, timestamp: Int)
```

   What do you have to modify?
   Implement `fee`, `gasCost`, `isValid`, and `describe` for it.

3. Go back to your Haskell module and do the same two extensions.
4. Look at your `groupByKind`:
   Can you give the return type `(List[Transfer], List[ContractDeploy], List[ContractCall])` instead of `(List[Transaction], List[Transaction], List[Transaction])`?
   Can you do the same in Haskell? What does this tell you about the difference between constructors and subtypes?

Follow-up: a Ledger class

The package functions above take `List[Transaction]` as an argument — the ledger is just a bare list. In a fully OO design, the ledger itself would be an object that encapsulates its data:

```
class Ledger(val txs: List[Transaction]):
  def totalFees: Double = ...
  def filterByAddress(addr: String): Ledger = ...
  def txInRange(lo: Int, hi: Int): Ledger = ...
  def groupByKind: (...) = ...
```

Refactor your package functions into methods on this `Ledger` class. Note that `filterByAddress` and `txInRange` now return a new `Ledger`, not a bare `List[Transaction]`.

Observations to discuss:

- Compare `Ledger(myTxs).totalFees` with `totalFees(myTxs)`. What does the class buy you?

- The Haskell module defines `type Ledger = [Transaction]` — a type alias, not a new type. How does this compare with the Scala class? Could you achieve something similar in Haskell using `newtype` ?

Part C — The Elixir perspective

Implement the transaction types in Elixir using structs:

```
defmodule Transfer do
  defstruct [:from, :to, :amount, :timestamp]
end

defmodule ContractDeploy do
  defstruct [:owner, :bytecode, :gas_limit, :timestamp]
end

defmodule ContractCall do
  defstruct [:caller, :contract, :function, :gas_limit, :timestamp]
end
```

Define `fee/1` , `gas_cost/1` , `is_valid/1` , and `describe/1` as functions in a `Ledger` module, using struct pattern matching.

Observations to discuss:

- Which style from the lecture (functional or OO) does the Elixir code resemble?
- Try passing a plain map `%{from: "x", to: "y", amount: 10, timestamp: 0}` (not a struct) to `fee/1` . What happens? Would the same mistake be caught at compile time in Haskell or Scala?
- Could you give `group_by_kind` a precise return type like Scala's `(List[Transfer], ...)` ? What does Elixir offer instead?